

SE & Cloud Computing course

03 - Behavioral Software (& AI!?) Engineering

Robert Feldt

Division of Software Engineering , Dept of Computer Science and Engineering
Chalmers University of Technology

robert.feldt@chalmers.se



@drfeldt



DAY TWO June 2, 2026

Time	Session
08:45 - 10:00	L3.1 Lecture: Behavioural SE (and AI?) Engineering (RF)
10:00 - 11:00	GL2 Guest lecture, Per Lenberg, <u>Hagnesia</u> : "Importance of Human Aspects in Software and AI/ML Engineering"
11:00 - 12:00	D3 Group debate: He et al., "LLM-based MAS for SE: Lit rev, vision, and road ahead"
12:00 - 13:15	Lunch
13:15 - 14:00	L3.2 Lecture: Second part of lecture 3
14:00 - 15:00	L4 Lecture: "Science versus Engineering" and ML Engineering (SE4ML) (RF)
15:00 - 15:20	Fika
15:20 - 16:00	Synthesis & Reflection: AI Engineering Manifesto!?
16:00 - 16:15	Assignment discussion and closing remarks

A person's eyes and hand are visible through the slats of white horizontal blinds. The person is looking directly at the camera, and their hand is visible on the left side, holding one of the slats open. The background is a solid white color.

Behavioral Software Engineering:

Let's focus on the humans...

Take home ideas from this lecture:

Behavioral Software Engineering:

People are not rational

**Thus, not enough to focus on better tech/
process & assuming their rational use.**

**Focusing directly on individuals, groups and
org can be at least or more effective.**

Take home ideas from this lecture:

Psychology:

It's complex, there are always many, inter-related factors

Psychology:

It's situational, i.e. things change as the situation/context changes

Let's look at one example!

**Professor of
Psychology, Princeton**

**Prospect theory,
Loss aversion,
Hedonic Psychology,
Heuristics & Biases**

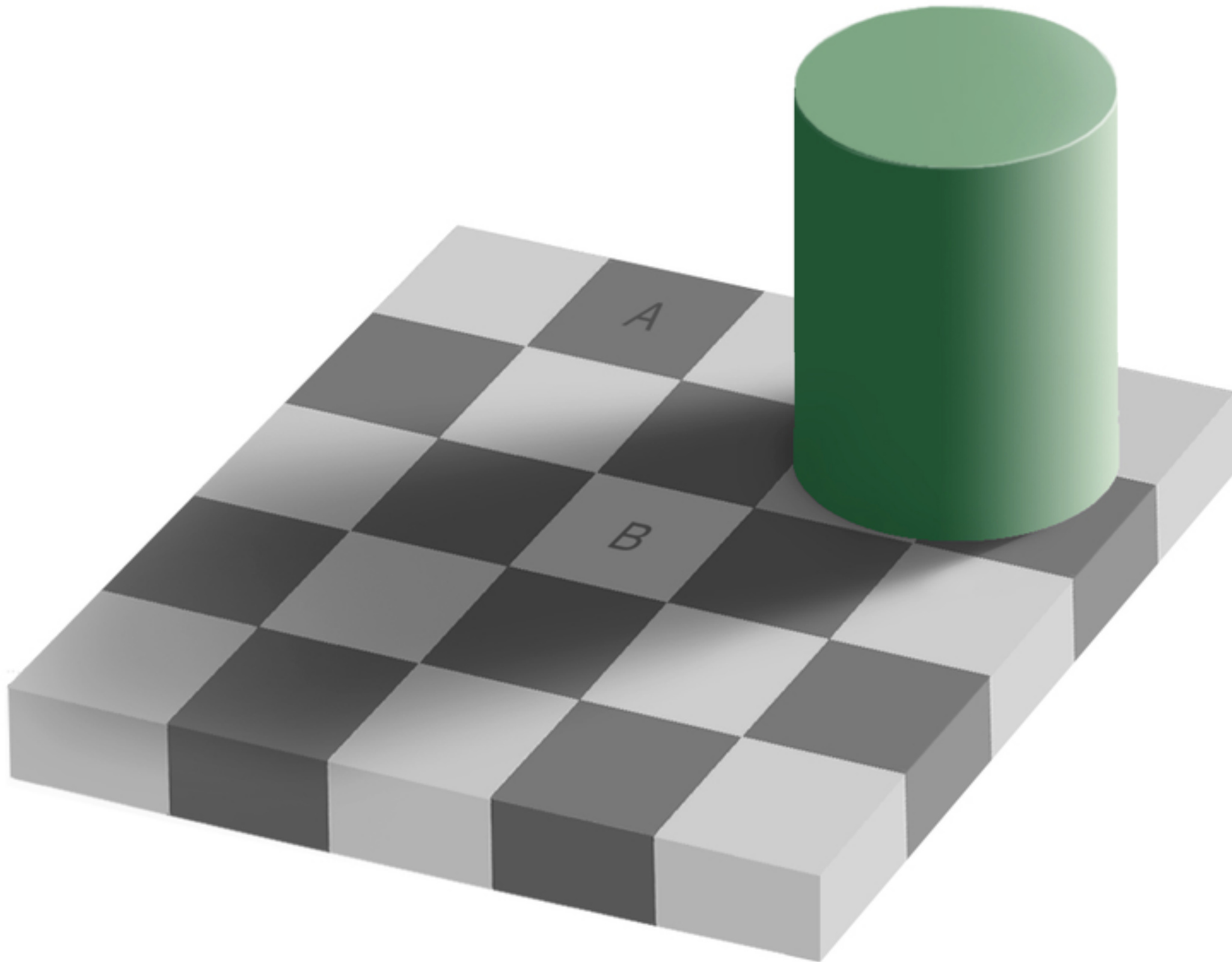
623,119 citations

H-index: 166

Daniel Kahneman

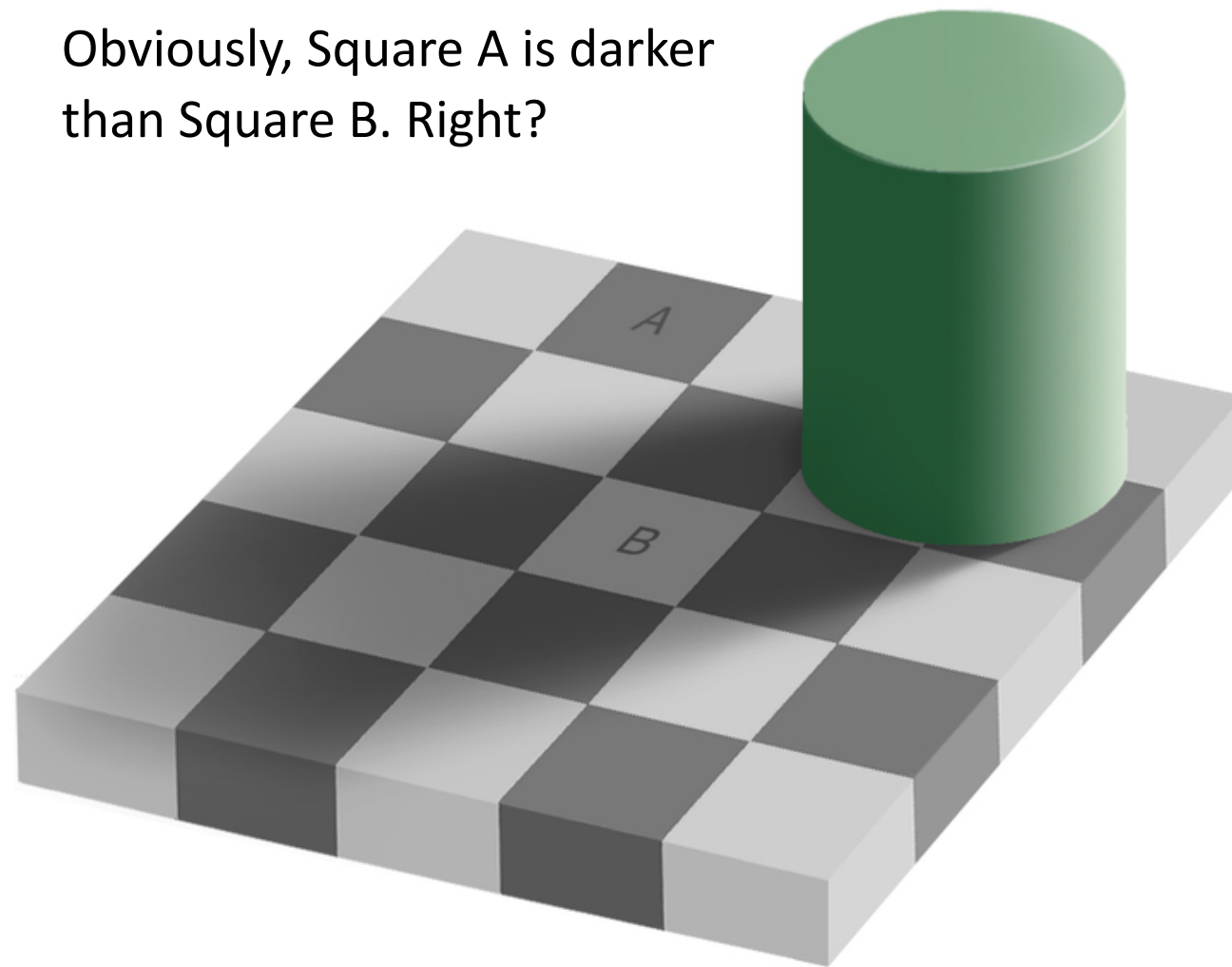


Square A is darker than Square B, right?



Even if you look again, *knowing* it's an illusion, the illusion's still real to you.

Obviously, Square A is darker than Square B. Right?



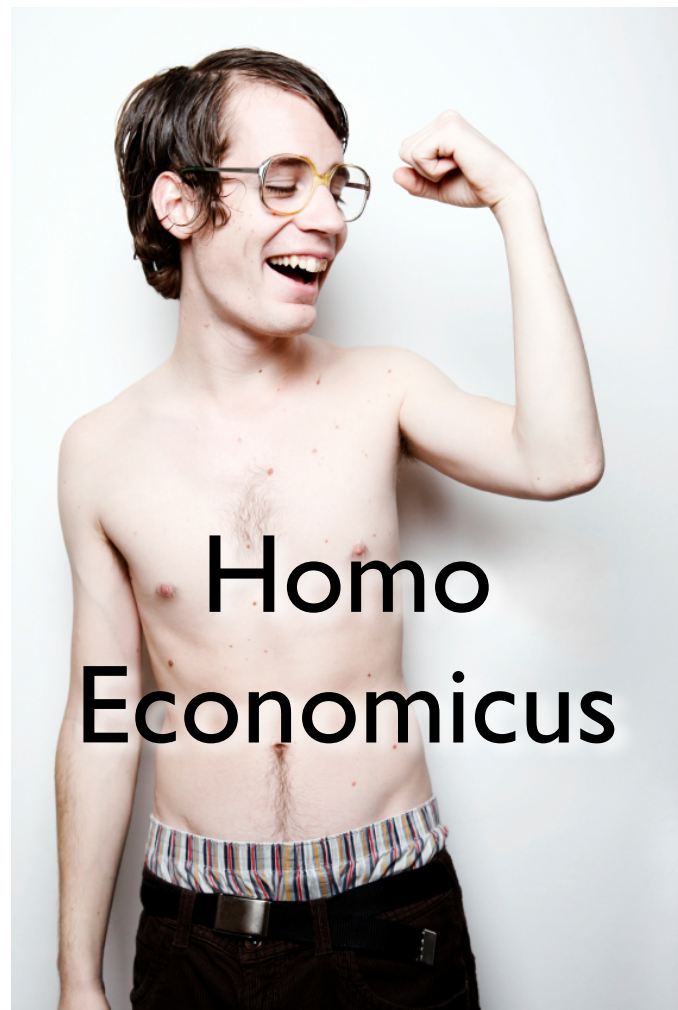
**Decades of research in
experimental psychology show:**

Same thing happens for other senses and
options, not only visual.

Cognitive illusions & biases are a
“feature” of our brains.

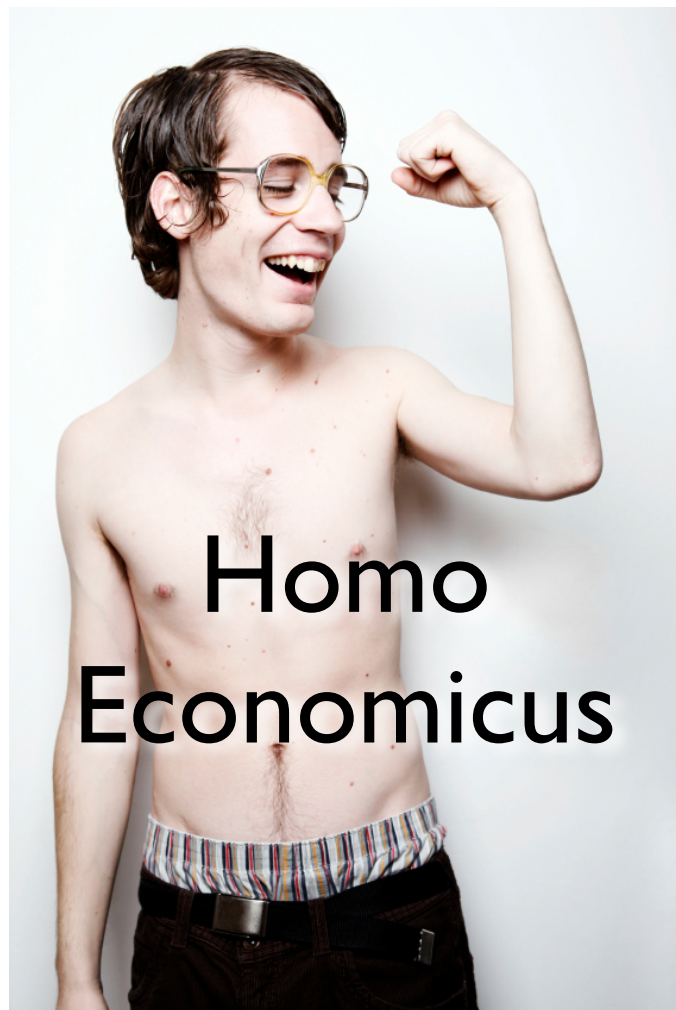
How could Kahneman get Nobel prize in Economics?

He has never taken even a basic course in economics.



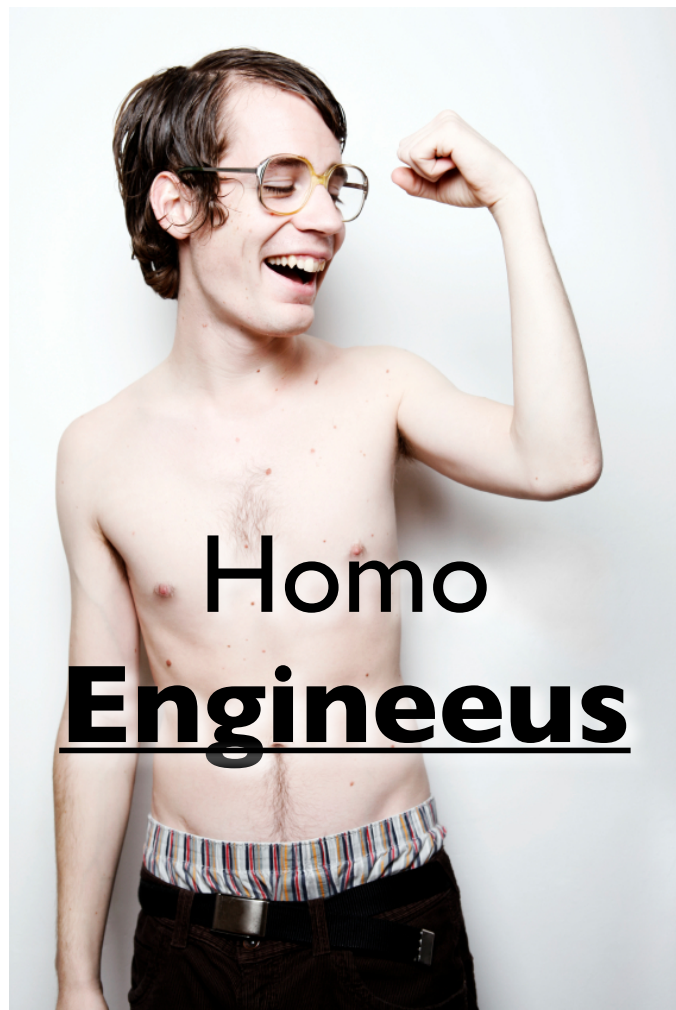
Behavioral Economics

“The study of psychology as it relates to the economic decision making activities of individuals, groups and institutions.”



Behavioral Software Engineering

*“The study of psychology as it relates to the **software engineering activities** of individuals, groups and institutions.”*



Trend in SE??

[All Databases](#) |
 [Select a Database](#) |
 [Web of Science](#) |
 [Additional Resources](#)

[Search](#) |
 [Author Search](#) |
 [Cited Reference Search](#) |
 [Advanced Search](#) |
 [Search History](#)

Topic Keywords	2012	2012	
Software Engineering	13,354	100 %	
Requirements	2173	16 %	
Design	4618	35 %	
Programming	2760	21 %	
Testing OR Verification	1349	10 %	
Organisation	102	0.7%	
Human Factors	90	0.7%	
Social OR Sociology	348	2.6%	
Psychology	68	0.5%	
Personality	29	0.2%	

→ 50.1%

→ 3.7%

Longer Trend in SE

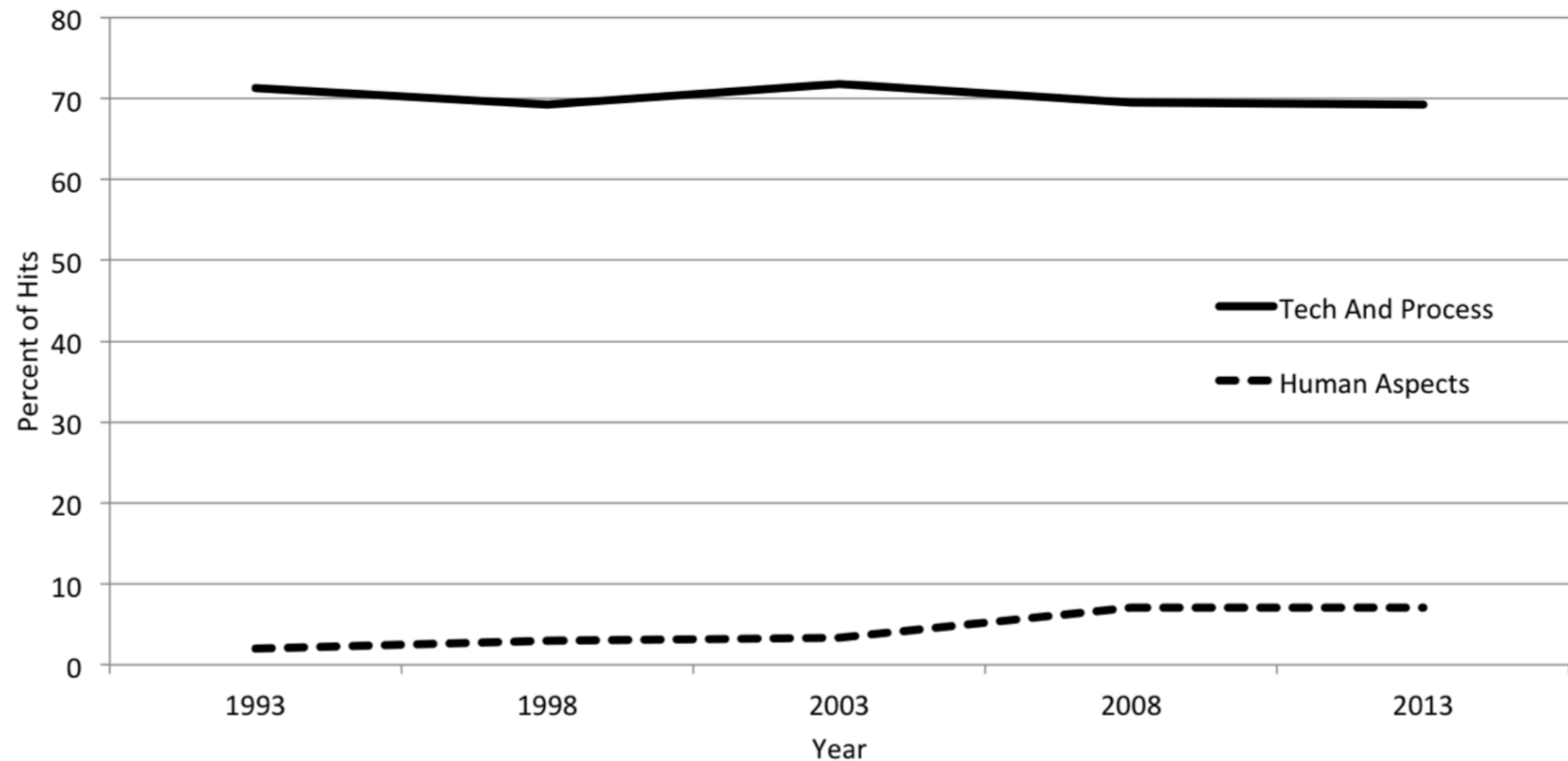


Figure 1: Percentage of software engineering journal articles 1993 to 2013 in ISI Web of Science that either (1) list a technology- or process-focused topic (red, full line, on top) and/or (2) list a human-focused topic (green, dashed line, on bottom).

BSE - The Definition

Behavioral Software Engineering (BSE) is the study of cognitive, behavioral and social aspects of software engineering performed by individuals, groups or organizations.

BSE - The Perspective

Behavioral Software Engineering (BSE), in order to better understand humans in software engineering, relies on the relatively mature scientific areas of Psychology and Social Science for concepts, theories and results about how to better understand and describe humans.

The Vision

*Make Software Engineering Change
Effective by Focusing on Humans*

Guiding implications

“A new process/method is not effective purely by how it reorders work but by how it affects the involved engineers and their communication”

“A new technology cannot affect industrial practice unless its effect on the engineers is understood and taken into account”

Most Extreme BSE Implication/Hypothesis

“Almost any new SE technology/process/method can work effectively in any industrial context if its effect on and alignment with the involved individuals and groups is understood and taken into account in its implementation.”

We asked industry:

Does the viewpoint taken in Behavioural Software Engineering differ from what you are used to? Or is it familiar to you?

Some answers from industry

*“Different perspective but similar to **communication and cultural issues** we discuss at work when it comes to outsourcing of testing”*

*“In distributed teams **it all comes down to the humans in the team**. I’ve been in teams in same corridor that took more energy/frustration to keep working than a team spread over the whole world.”*

*“**SCRUM defines BSE-style values**, such as focus, courage, openness, commitment and respect. But an organisation promoting SCRUM but working in a traditional hierarchical way, will probably fail at some point or at least be not as efficient.”*

*“BSE has another viewpoint than I’m used to. For example, when a new process has been introduced, the viewpoint has been what new deliveries are needed, how is work progress measured etc. But very little about how the changes **will affect the individuals well-being or the group dynamics**.”*

3 units of analysis in BSE

Organisation

Work/Org Psychology



Group

Social Psychology

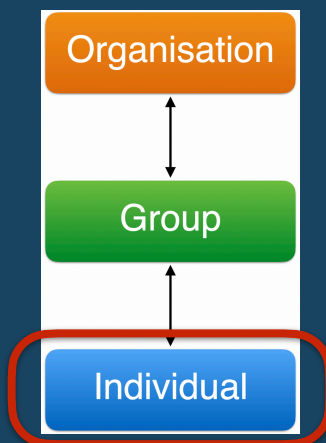


Individual

Psychology

Cognitive bias summary

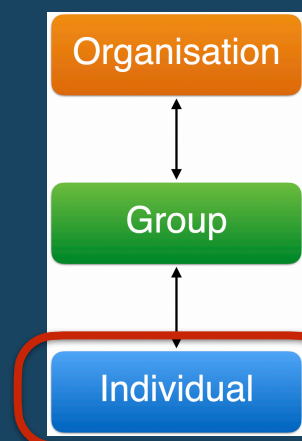
- We need heuristics (mental shortcuts), but these also lead to
 - recurring errors in thinking, or patterns of bad judgment observable in different people and contexts (i.e. cognitive biases).
- Confirmation bias—the tendency to pay more attention to information that agrees with our preconceptions.
- Availability bias refers to a tendency of being influenced by the information that is easy to recall and by the information that is recent or widely publicized.
- Anchoring bias – to be influenced by available but irrelevant information.



Cognitive bias summary

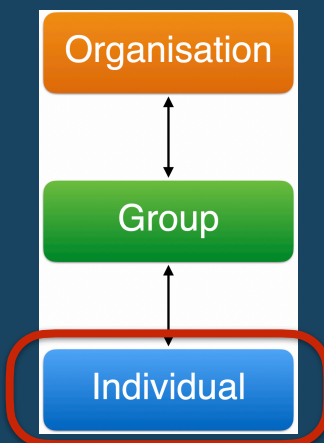
- **Confirmation bias effects in SE**

- Incorrect beliefs about the completeness and correctness of answers or solutions [P13]
- Insufficiently exploring code when dealing with a change request [P18]
- Rejecting results and measurements that do not support the analyst's beliefs [P37]
- Higher defect rate [P4]
- More post-release defects [P6]
- Misidentifying causes of program failure [P32]
- Unrealistic trials of SE tools or techniques [P26]
- Running more tests that show a system works than tests that find problems [P31], [P32]



Cognitive bias mitigation

- **Confirmation bias mitigation strategies**
 - Traceability [P16]
 - Explicitly exploring disconfirmatory evidence [P32]
 - Generating multiple solutions [P37]



Cognitive biases in developing biased Artificial Intelligence recruitment system

Melika Soleimani
School of Management
Massey University
m.soleimani@massey.ac.nz

Ali Intezari
UQ Business School
University of Queensland
a.intezari@business.uq.edu.au

Nazim Taskin
Management Information Systems
Department
Bogazici University
nazim.taskin@boun.edu.tr

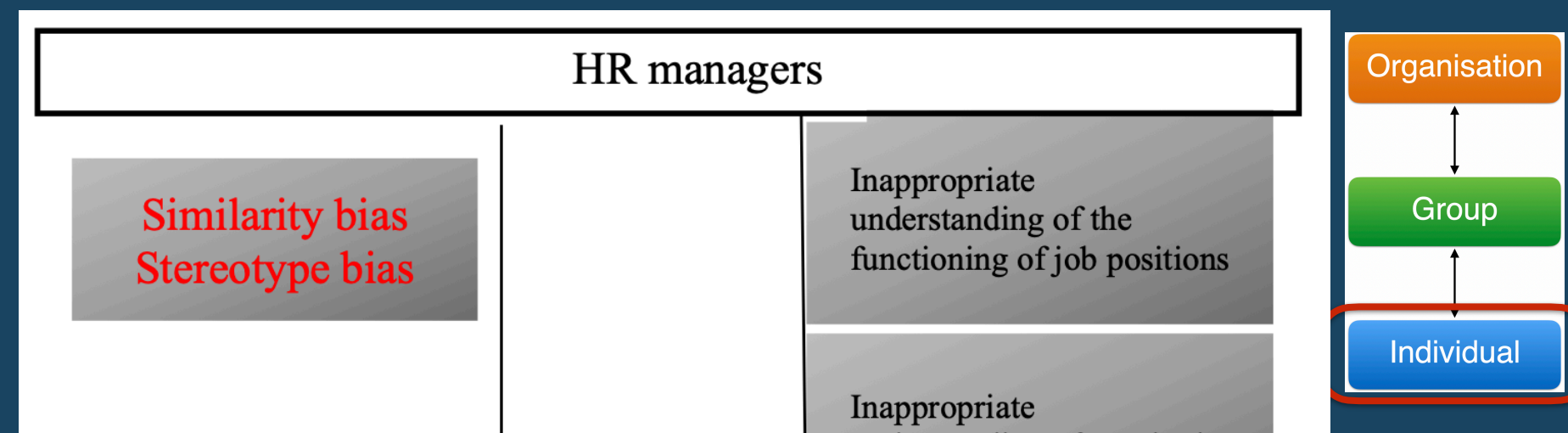


- **HR Managers:**

- Stereotype bias - “People working for company C are good so this one must be to”
- Similarity bias - Hiring panels tend to select candidates similar to themselves
- Inappropriate understanding of position & required soft skills

- **AI Engineers:**

- Feature selection biased by specific HR manager
- Reliance on inadequate information



Mitigating Cognitive Biases in Developing AI-Assisted Recruitment Systems: A Knowledge-Sharing Approach

Melika Soleimani, Massey University, New Zealand

Ali Intezari, University of Queensland, Australia

 <https://orcid.org/0000-0001-9369-2859>

David J. Pauleen, Massey University, New Zealand

 <https://orcid.org/0000-0003-4412-0162>

ABSTRACT

Artificial intelligence (AI) is increasingly embedded in business processes, including the human resource (HR) recruitment process. While AI can expedite the recruitment process, evidence from the industry, however, shows that AI-recruitment systems (AIRS) may fail to achieve unbiased decisions about applicants. There are risks of encoding biases in the datasets and algorithms of AI which lead AIRS to replicate and amplify human biases. To develop less biased AIRS, collaboration between HR managers and AI developers for training algorithms and exploring algorithmic biases is vital. Using an exploratory research design, 35 HR managers and AI developers globally were interviewed to understand the role of knowledge sharing during their collaboration in mitigating biases in AIRS. The findings show that knowledge sharing can help to mitigate biases in AIRS by informing data labeling, understanding job functions, and improving the machine learning model. Theoretical contributions and practical implications are suggested.

Five-factor models, Big Five, OCEAN



Openness



Is open
to novel
experience.

Has
narrow
interests.



Conscientiousness



Is responsible
and
dependable.

Is impulsive
and careless.



Extraversion



Is outgoing
and
decisive.

Is retiring
and
withdrawn.



Agreeableness



Is warm
and good-
natured.

Is unfriendly
and cold.



Neuroticism



Is stable
and not a
worrier.

Is nervous
and
emotionally
unstable.

OCEAN

Each letter in the word OCEAN is the first letter of one of the Big Five traits.

[Plotkin2011]

The Connection Between Burnout and Personality Types in Software Developers

Emanuel Mellblom, Carmenta Geospatial Technologies

Isar Arason, Independent Interactive Media Consultant

Lucas Gren and Richard Torkar, Chalmers University of Technology
and the University of Gothenburg

// This article examines the connection between the five-factor model personality traits and burnout in software developers and aims to validate generalizations of findings in other fields. //

STRESS IS A common occurrence in many professions. It is a natural response to adverse circumstances, heightened alertness, and improved reaction speed and allows a person to better handle a trying situation. However, this emotional state takes a toll on the human body, impacting the mental and physical health of a person if sustained for long periods of time.¹

People who experience long-term stress on the job are at greater risk of burnout, which occurs when a person can no longer handle the stress put on them, resulting in reduced work efficiency, unhappiness, and health problems. Some people, however, appear to thrive in high-intensity environments for long periods of time without burnout.¹ This may be related to the apparent link between personality types and stress suggested in other studies.² We note that personality is one aspect of a person, and other aspects may be more important to avoid stress, e.g., social support.³

A form of personality measurement is the five-factor model (FFM) also called the *Big Five*. The FFM is a construct composed of five personality types: openness to experience, conscientiousness, extraversion, agreeableness, and neuroticism.⁴ Our goal is to investigate the connection between the FFM personality types

Burnout+Personality conclusions:

- 1. First study to research the connection between FFM personality types and burnout in software development
- 2. Neuroticism is a strong indicator of workplace burnout in software developers, confirming previous studies in other domains
- 3. The other FFM personality traits (Openness, Conscientiousness, Extraversion, and Agreeableness) did not add power to the model
- 4. Did **not** investigate the quality of work in connection to personality or take any other confounding factors into account like teamwork
- 5. Employers could be aware of and support software developers with high neuroticism
- **Limitations:**
 - Small sample, self-selected/voluntary sample in online study, response rate unclear

Motivation

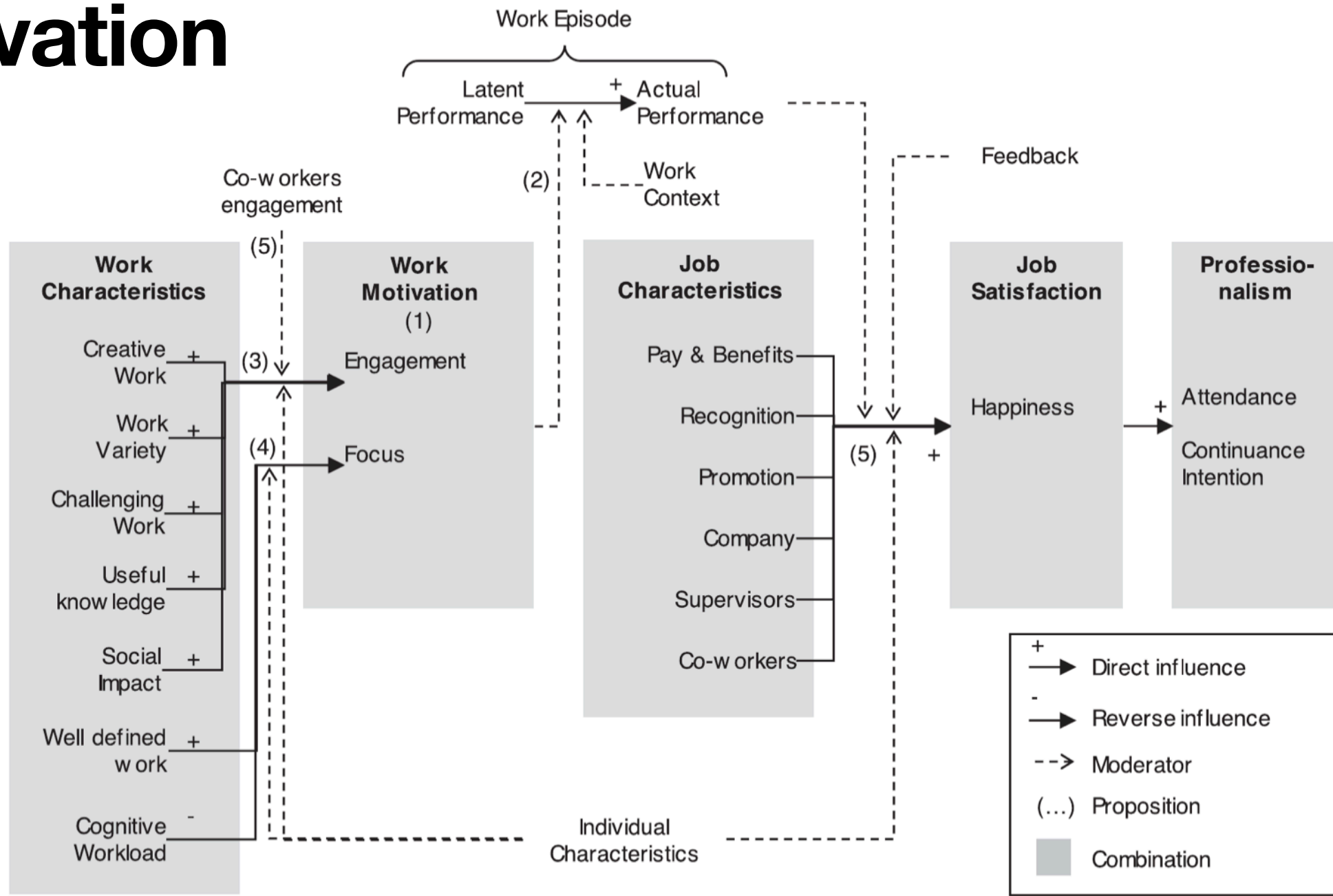
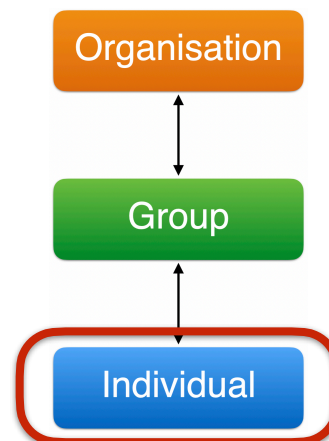


Fig. 7. Model of work motivation and job satisfaction.



Group discussion: He et al



BAIE!?! - The Definition

*Behavioral **AI** Engineering (BAIE) is the study of cognitive, behavioral and social aspects of **AI** engineering performed by individuals, groups or organizations.*

BAIE: AI Changes Developer Behaviour

- AI coding tools reshape individual cognition & behavior

Grounded Copilot: How Programmers Interact with Code-Generating Models

SHRADDHA BARKE*, UC San Diego, USA

MICHAEL B. JAMES*, UC San Diego, USA

NADIA POLIKARPOVA, UC San Diego, USA

Powered by recent advances in code-generating models, AI assistants like Github Copilot promise to change the face of programming forever. But what *is* this new face of programming? We present the first grounded theory analysis of how programmers interact with Copilot, based on observing 20 participants—with a range of prior experience using the assistant—as they solve diverse programming tasks across four languages. Our main finding is that interactions with programming assistants are *bimodal*: in *acceleration mode*, the programmer knows what to do next and uses Copilot to get there faster; in *exploration mode*, the programmer is unsure how to proceed and uses Copilot to explore their options. Based on our theory, we provide recommendations for improving the usability of future AI programming assistants.

Additional Key Words and Phrases: Program Synthesis, AI Assistants, Grounded Theory

BAIE: AI Changes Developer Behaviour

Grounded Copilot

How programmers interact with code-generating models

Barke, James & Polikarpova • OOPSLA 2023

Study at a glance



Grounded-theory user study



20 participants



4 languages: Python, Rust, Haskell, Java



Tasks across diverse programming scenarios



5 livestreams used to reinforce the findings

Core finding

Copilot use is bimodal

Programmers switch between two interaction modes



Acceleration mode



Known goal: the programmer already knows what to do



Copilot acts like fast autocomplete



Mostly code/context prompts

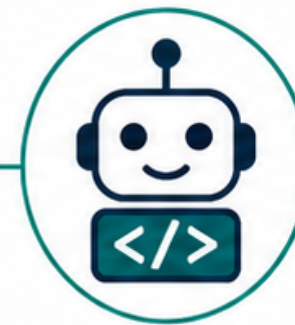


Short suggestions preserve flow



Quick accept/reject, lighter validation

Code prompts most common: 71.4%



Exploration mode



Uncertain goal: the programmer is not sure how to proceed



Copilot helps explore options or get a starting point



More comment prompts and multi-suggestion use



Slower, deliberate interaction



More validation: run code, inspect output, read docs

Comment prompts much higher:
57.2% vs 13.1%

fast / flow

slow / deliberate



Design implications

- 1 Make the assistant mode-aware
- 2 In acceleration: short, high-confidence, low-interruption suggestions
- 3 In exploration: compare alternatives and control context
- 4 Support always-on validation and testing



Limits of the study

- 1 Small, lab-based sample
- 2 Focused mainly on code authorship
- 3 Best seen as a useful lens, not a universal taxonomy



Main takeaway: AI coding assistants help differently when developers are accelerating known work versus exploring uncertain work.

BAIE: AI Changes Developer Behaviour

4.1 Acceleration

Acceleration is characterized by the programmer being “in the zone” and actively “driving” the development, while occasionally relying on Copilot to complete their thought process. A programmer will often accept a Copilot suggestion without much comment and keep on going without losing focus. In this interaction mode, programmers tend to think of Copilot as an intelligent autocomplete that just needs to complete their line of thought. This idea was well put by P13 who said:

“I think of Copilot as an intelligent autocomplete... I already have the line of code in mind and I just want to see if it can do it, type it out faster than I can.”

“I have a picture in mind and that picture ranges from syntactic or textual features—like a literal shape in words—to semantic about the kind of methods that are being invoked, the order in which they should be invoked, and so on. When I see a suggestion, the closer that suggestion is to the mental image I hold in my head, the more likely I am to trust it.”

BAIE: AI Changes Developer Behaviour

4.2 Exploration

In the previous section we focused on the use of Copilot when the programmer has a good idea for how to approach the task. But what if they do not? In that case they might use Copilot to help them get started, suggest potentially useful structure and API calls, or explore alternative solutions. All of these behaviors fit under what we call exploration mode. Exploration is characterized by the programmer letting Copilot "drive", as opposed to acceleration, where the programmer is the driver. In the rest of this section, we first describe the conditions that lead programmers to enter exploration mode, and then we characterize the common behaviors in that mode.

“Copilot feels useful for doing novel tasks that I don’t necessarily know how to do. It is easier to jump in and get started with the task”.

“I think that the natural language prompt is more cohesive because it’s interruptive to be typing out something and then for Copilot to guess what you’re thinking with that small pseudocode. It’s nice to have a comment that you’ve written about your mental model and then going to the next line and seeing what Copilot thinks of that.”

BAIE: AI Changes Developer Behaviour

A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges

Jenny T. Liang
Carnegie Mellon University
Pittsburgh, PA, USA
jtliang@cs.cmu.edu

Chenyang Yang
Carnegie Mellon University
Pittsburgh, PA, USA
cyang3@cs.cmu.edu

Brad A. Myers
Carnegie Mellon University
Pittsburgh, PA, USA
bam@cs.cmu.edu

ABSTRACT

The software engineering community recently has witnessed widespread deployment of AI programming assistants, such as GitHub Copilot. However, in practice, developers do not accept AI programming assistants' initial suggestions at a high frequency. This leaves a number of open questions related to the usability of these tools. To understand developers' practices while using these tools and the important usability challenges they face, we administered a survey to a large population of developers and received responses from a diverse set of 410 developers. Through a mix of qualitative and quantitative analyses, we found that developers are most motivated to use AI programming assistants because they help developers reduce key-strokes, finish programming tasks quickly, and recall syntax, but resonate less with using them to help brainstorm potential solutions. We also found the most important reasons why developers do *not* use these tools are because these tools do not output code that addresses certain functional or non-functional requirements and because developers have trouble controlling the tool to generate the desired output. Our findings have implications for both creators and users of AI programming assistants, such as designing minimal cognitive effort interactions with these tools to reduce distractions for users while they are programming.

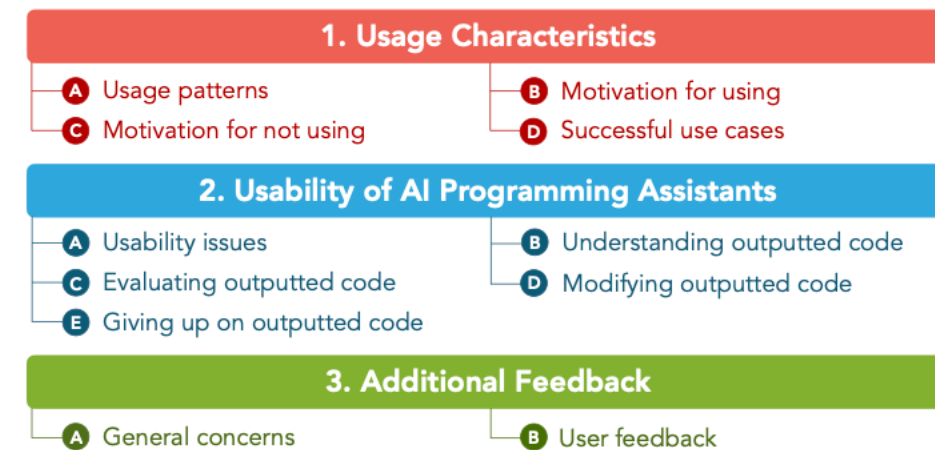


Figure 1: An overview of the topics covered in our usability study of AI programming assistants.

1 INTRODUCTION

The recent widespread deployment of AI programming assistants, such as GitHub Copilot [6] and ChatGPT [1], has introduced a new paradigm to building software that has taken the software engineering community by storm. Some current publications report that AI programming assistants are powerful enough to produce high-quality code suggestions for developers [59, 61]. While some

Usability of AI Programming Assistants

Liang, Yang & Myers · ICSE 2024

A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges

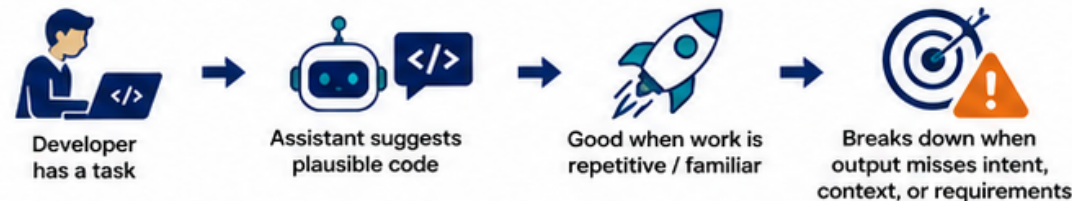
Study at a glance

- 1 410 developers surveyed
- 2 ~4% response rate
- 3 Recruited from Copilot / Tabnine-related GitHub activity
- 4 Survey deployed: Jan 2023
- 5 GitHub Copilot users reported a median of 30.5% of code written with its help

Core finding

Useful for acceleration — limited by **control** and **alignment**

AI assistants help most when developers already know roughly what they want, but struggle when intent, context, and requirements must be communicated precisely.



Deep usability bottleneck

control

traceability

code context

functional +
non-functional
requirements

**Best seen as a low-friction collaborator,
not an autonomous problem solver.**

Why developers use them

- | | |
|---------------------------------|--------------|
| Autocomplete / fewer keystrokes | 86% |
| Finish tasks faster | 76% |
| Recall syntax or APIs | 68% |
| Repetitive code | 78× mentions |
| Simple logic / small utilities | 68× mentions |

Other successful uses

QA / test cases

proof-of-concepts

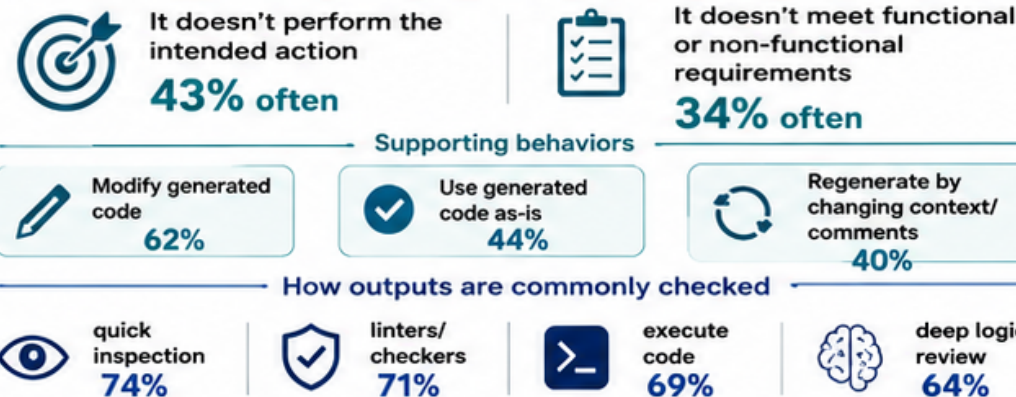
learning

documentation

Top usability challenges

- | | |
|--|-----------|
| 1 Not knowing what input influenced the output | 30% often |
| 2 Giving up and writing the code manually | 28% often |
| 3 Trouble controlling the tool | 26% often |
| 4 Suggestions feel distracting | 23% often |
| 5 Trouble evaluating correctness | 23% often |

Why users give up on generated code



Design implications

- 1 Prioritize low-cognitive-effort interaction
- 2 Give developers more control over output
- 3 Show what context influenced the suggestion
- 4 Support functional and non-functional requirements
- 5 Allow lightweight feedback / personalization

Limits

- Self-reported survey data
- Enthusiast-biased sample
- Early-2023 tools, so exact percentages may age quickly

Main takeaway: AI programming assistants are already useful, but mainly as acceleration tools. Their biggest usability challenge is controllable, context-aware alignment with developer intent and project requirements.

BAIE: AI Changes Developer Behaviour

Do Users Write More Insecure Code with AI Assistants?

Neil Perry*

Stanford University

Megha Srivastava*

Stanford University

Deepak Kumar

Stanford University / UC
San Diego

Dan Boneh

Stanford University

ABSTRACT

AI code assistants have emerged as powerful tools that can aid in the software development life-cycle and can improve developer productivity. Unfortunately, such assistants have also been found to produce insecure code in lab environments, raising significant concerns about their usage in practice. In this paper, we conduct a user study to examine how users interact with AI code assistants to solve a variety of security related tasks. Overall, we find that participants who had access to an AI assistant wrote significantly less secure code than those without access to an assistant. Participants with access to an AI assistant were also more likely to believe they wrote secure code, suggesting that such tools may lead users to be overconfident about security flaws in their code. To better inform the design of future AI-based code assistants, we release our user-study apparatus and anonymized data to researchers seeking to build on our work [at this link](#).

CCS CONCEPTS

risks of AI assistants in the context of how developers choose to use them. Such work is important in order understand the practical security challenges introduced by AI-powered code-assistants and the ways users prompt the AI systems to inadvertently cause security mistakes.

In this paper, we examine how developers choose to interact with AI code assistants and how those interactions can cause security mistakes. To do this, we designed and conducted a comprehensive user study where 47 participants conducted five security-related programming tasks spanning three different programming languages (Python, JavaScript, and C). Our study is driven by three core research questions:

- **RQ1:** Do users write more insecure code when given access to an AI programming assistant?
- **RQ2:** Do users trust AI assistants to write secure code?
- **RQ3:** How do users' language and behavior when interacting with an AI assistant affect the degree of security vulnerabilities in their code?

Insecure Code with AI Assistants

Do Users Write More Insecure Code with AI Assistants?

Perry, Srivastava, Kumar & Boneh · CCS 2023

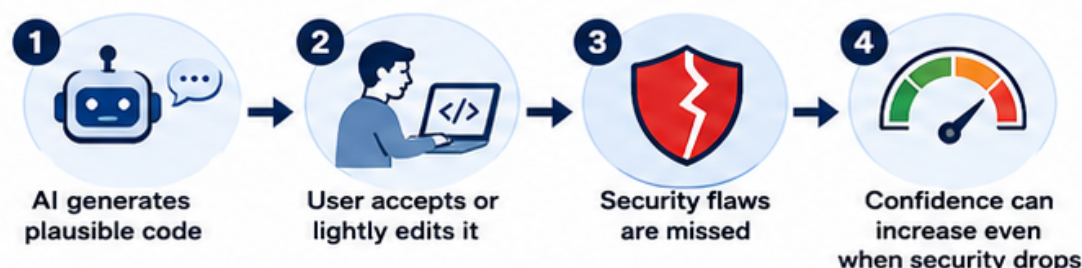
Study at a glance

- Randomized user study
- 47 participants:
33 with AI assistant, 14 control
- 5 security-related programming tasks
- 3 languages:
Python, JavaScript, C
- Outcomes analyzed:
code security, correctness, trust, prompts

Core finding

AI assistance often reduced security

The main risk is at the human-tool boundary



Why this happens: contributing factors



87% of secure AI-assisted answers required significant edits

Why this matters

- Code can look correct yet still be insecure
- AI can automate false assurance, not just code generation
- Security burden shifts from writing code to validating it
- Human oversight is not enough if the user lacks security expertise

Design implications

- 1 Add guardrails, warnings, and safer defaults
- 2 Support validation: tests, static analysis, and library guidance
- 3 Encourage security-aware prompting and active editing
- 4 Do not treat the assistant as automatically trustworthy
- 5 Train and tune assistants on more secure code

Key evidence

- Across 4 of 5 tasks, participants with AI wrote **less secure code**
- Q1 Encryption**: AI group significantly more likely to write insecure code ($p = 0.017$)
- Q2 Signing**: secure solutions **3%** with AI vs **21%** in control
- Q4 SQL**: vulnerable to SQL injection **36%** with AI vs **7%** in control
- Q5** was the main exception
- Participants with AI often believed their answers were **more secure**
Users with secure solutions tended to trust the AI less than users with insecure solutions

Security tasks



Limits of the study

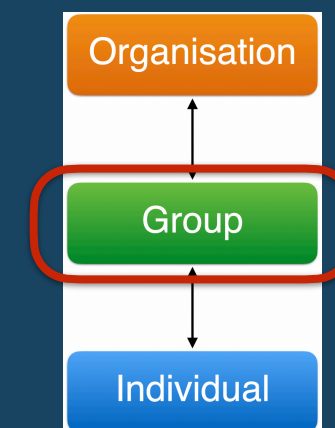
- Mostly university students
- Lab tasks, not real long-term development
- Older Codex-based assistant and UI
- Small sample, so exact rates should not be overgeneralized



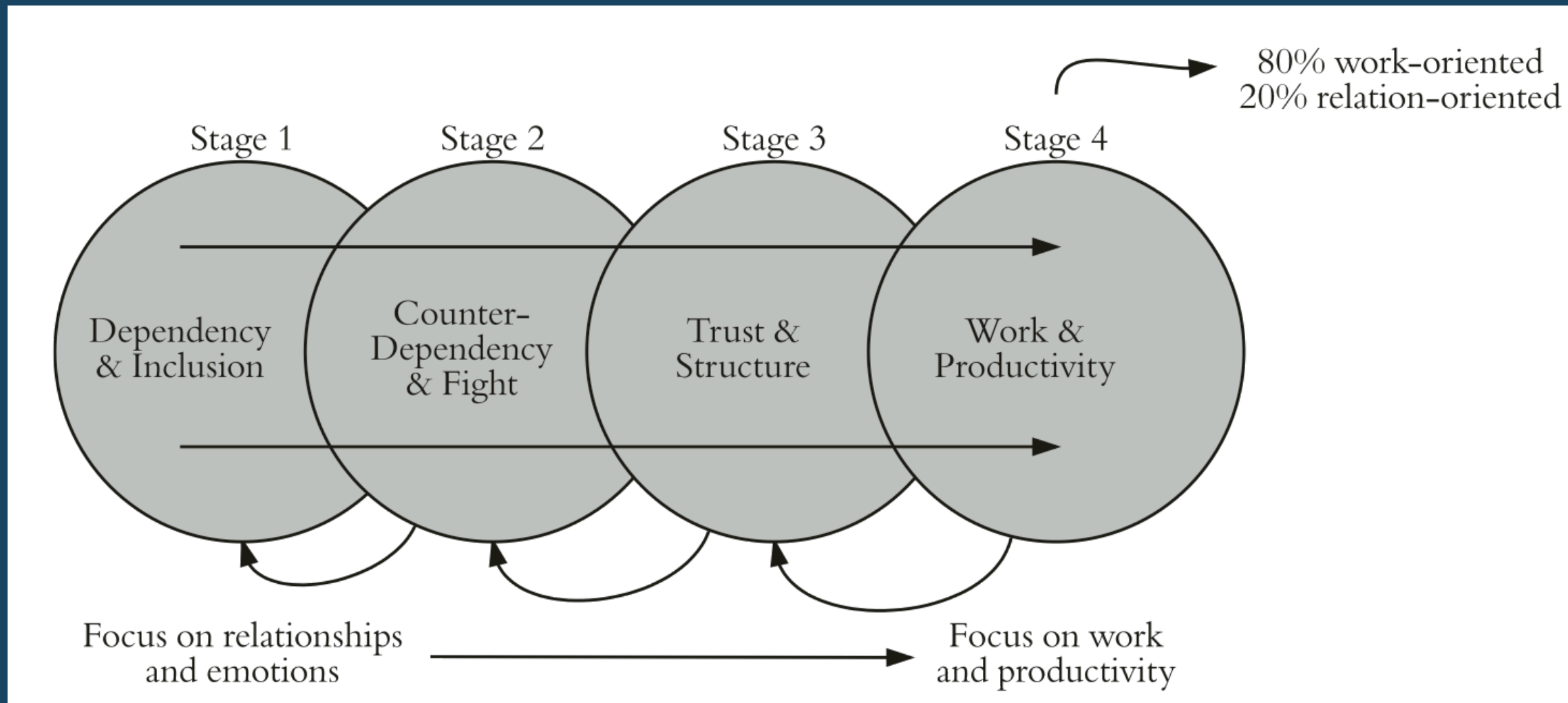
Main takeaway: AI coding assistants can make code less secure while making developers feel more confident — so safer AI coding needs validation, guardrails, and informed users.

Group Level BSE concepts

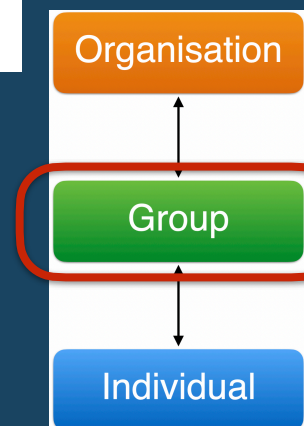
- Group Dynamics and Development
- Psychological Safety
- Norm Clarity
- Group cohesion & polarisation - reinforces majority/extreme view
- Groupthink - group agreement vs good decision
- Creativity in groups, creativity techniques
- Social sensitivity & Collective Intelligence
- Team diversity & Gender issues



Group Dynamics: The Integrated Model of Group Development



Connection to agile teams



Psychological Safety and Norm Clarity in Software Engineering Teams

Per Lenberg
Div. of Software Engineering
Chalmers and University of Gothenburg
perle@chalmers.se

Robert Feldt
Div. of Software Engineering
Chalmers and University of Gothenburg
robert.feldt@chalmers.se

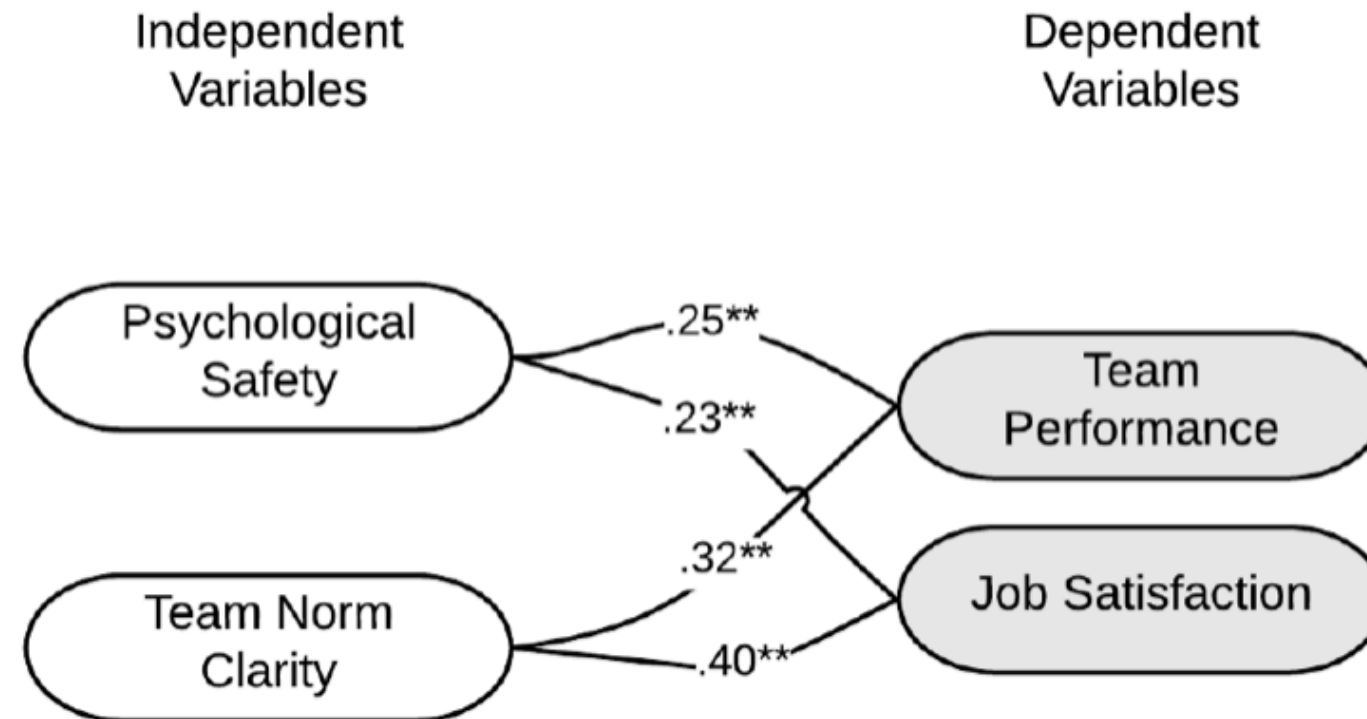
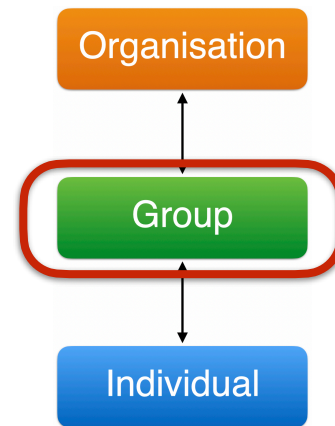
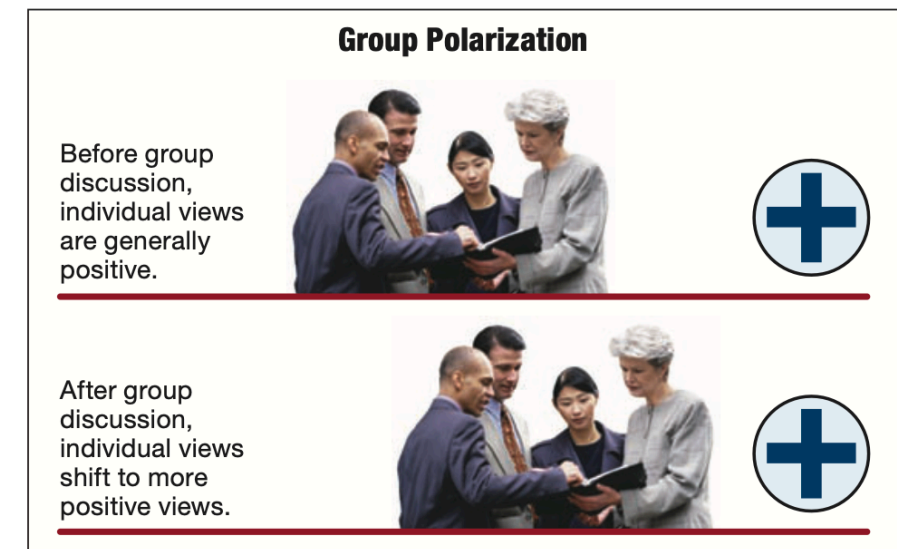


Figure 2: Model used to predict *team performance* and *job satisfaction*. The connection line figures are the beta values from Table **5** and **6**. '**' indicates that the value was statistically significant at .01.

Definitions: Group cohesion, polarisation, Groupthink etc [Plotnik2011]

- “**Group cohesion** is degree to which the **members** of a group **desire to remain in the group.**”
- “**Group polarization** is a phenomenon in which **group discussion reinforces the majority’s point of view** and **shifts** that view to a **more extreme position.**”
- “**Groupthink** refers to a **group making bad decisions** because the group is **more concerned about reaching agreement and sticking together** than gathering the relevant information and considering all the alternatives.”
- “**Collective intelligence** is a measure of the general effectiveness of a group on a wide range of tasks.”



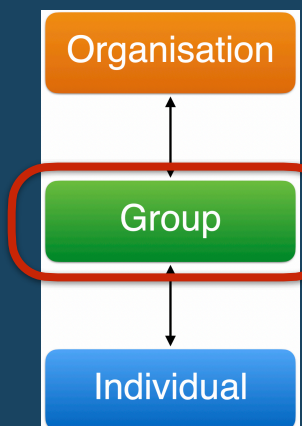
Social Sensitivity & Collective Intelligence (CI)

- Several predictors of collective intelligence has been found:
 - “Average and maximum intelligence of individual group members were correlated with CI, but **only moderately so**. In other words, having a lot of smart people in a group did not necessarily make a smart group.”
 - “**Significant correlation between CI and the average ToM** scores of group members, as measured by the “Reading the Mind in the Eyes” test”
 - “CI was **negatively correlated** with the variance in the number of speaking turns by group members. In other words, groups where a few people dominated the conversation were less collectively intelligent than those with a more equal distribution of conversational turn taking”
 - “CI was **significantly correlated with the proportion of females** in the group, with groups having more females being more collectively intelligent.”

[Engel2014]

Diversity: Team diversity

- *Team diversity* refers to the individual differences between members of a team. It can be present on various dimensions such as value diversity (e.g., beliefs, goals, values), information diversity (e.g., experience, knowledge, background), and social diversity (e.g., gender, age, race) (Rodríguez-Pérez, 2021)



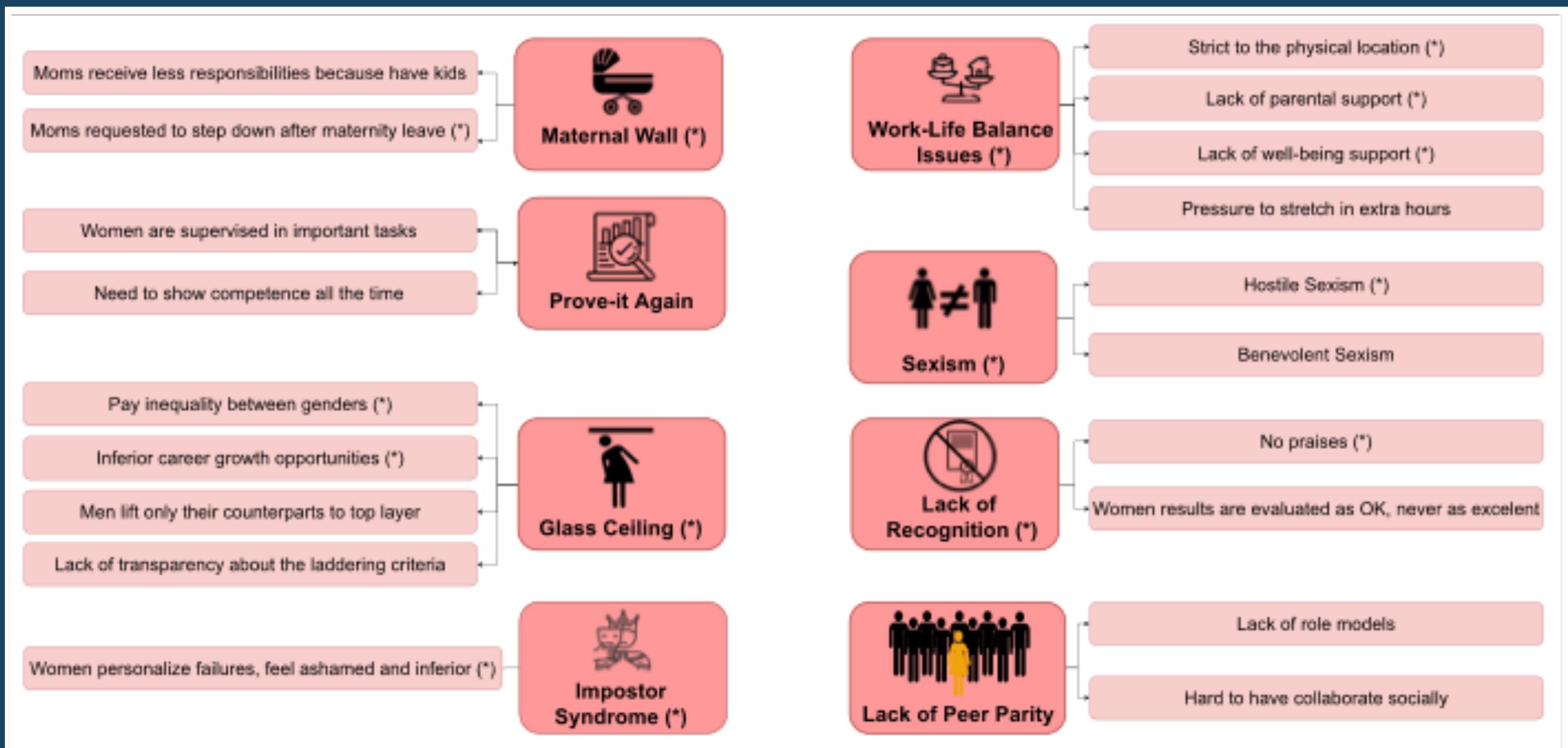
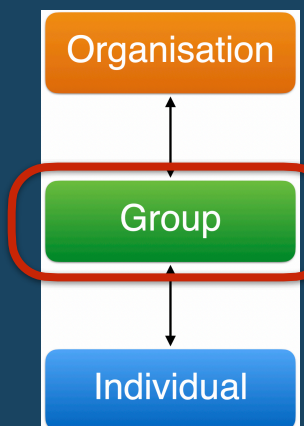


Figure 1: The challenges reported by women who participated in our study. Those marked with an asterisk (*) were reported as a challenge that ultimately can lead women to leave the company.

Org Level BSE concepts

- Organisational change/improvement models
- Stress, turnover, and sustainability
- Politics and Power
 - Example: Intentional Distortions in Software Development



Kotter's 8-step Organisational Change Model



(Source: Adapted from Kotter 1996)

Types of organisational power [Norbom2016]

- Raven and French's 5 traditional sources of social power:
 - **Reward**: administer desirable outcomes (rewards e.g. pay, promotion)
 - **Coercive**: to give or recommend punishments (reprimands, undesirable work tasks...)
 - **Legitimate**: from formal position, ppl comply out of obligation/responsibility
 - **Expert**: from superior or special knowledge or skill => respect/credibility
 - **Referent**: based on individual characteristics => attention/respect/admiration
- More recent additions:
 - **Connection** power: stemming from connections/networking to important/influential individuals
 - **Information** power: ability to convince with clear, logical, convincing argument
 - Less empirical support for the latter, but much support for the former
 - Information power seems to overlap heavily with Expert power

“Formal”

“Informal”

Ideal
Estimate

Raw
Estimate

Distorted
Estimate

Actual
Outcome

+

Pessimism

Hiding other activities
Ensure no overspending
Job securing
Avoid unnecessary functionality

Lack of Req Understanding
Lack of Experience

Inaccurate/no feedback/
learning

Prep for negotiations
Personal agendas (career etc)
Organizational agendas
Budget => Estimate
Point estimates

■

Optimism

Ordering of unrealistic est.
“Selling” ideas

Overlooked/misunderstood risks/tasks

From Challenge to Change: A Human-Centric Framework of Actionable Guidelines for AI Transformations

Master's Thesis in Computer Science and Engineering

Theocharis Tavantzis

Department of Computer Science and Engineering
CHALMERS UNIVERSITY OF TECHNOLOGY
UNIVERSITY OF GOTHENBURG
Gothenburg, Sweden 2025

BSE-related challenge	BSE-related sub-challenge	Summary
Change Management Strategy	External communication	This theme refers to the challenges, when it comes to communicating the change.
	Internal communication	
Ethical Concerns	Competitive gap among the market	The theme of <i>Ethical Concerns</i> refers to ethical issues that arise during the AI-driven Organizational Change.
	Considerate use of external AI with company's data	
	Risk aversion due to sensitive data	
Organizational Readiness for AI Adoption	Adaptation to global and local data legislation	The <i>Organizational Readiness for AI Adoption</i> theme relates to the importance of managing organizational issues, e.g. diverse views and sector-specific complexities, for a smooth AI adoption.
	Cross-Functional Collaboration	
	Ensuring organizational alignment	
	Handling differing views regarding AI transformation	
	Inter-generational Collaboration	
	Public sector's unique nature	
Resistance to Change	Job displacement	The <i>Resistance to Change</i> theme concerns common fears and conditions, which collectively create obstacles to the AI-driven Organizational Change.
	Lack of deep AI knowledge and understanding	
	Lack of patience to work with AI	
Skills in Era of AI	Keeping up with AI advancements	That particular theme focuses on the need for skill adaptation and continuous learning together with the adoption of AI.
	Lack of skills due to AI	
	Necessity to adjust skillset	
Strategic Adoption of AI	Balance automation with human supervision	The <i>Strategic Adoption of AI</i> theme summarizes the need of integrating AI in a way that will complement the human skills.
	Considering AI as a team member, instead of tool	
	Contextual Use of AI	
	Establish balance in using internal and external AI solutions	
	Over-reliance on AI	
	Purpose-driven AI adoption	

Table 3.1: BSE-related challenges and sub-challenges within AI integration [129]

BAIE at the group/ org levels

Collaboration Challenges in Building ML-Enabled Systems: Communication, Documentation, Engineering, and Process

Nadia Nahar

nadian@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Grace Lewis

Carnegie Mellon Software Engineering Institute
Pittsburgh, PA, USA

Shurui Zhou

University of Toronto
Toronto, Ontario, Canada

Christian Kästner

Carnegie Mellon University
Pittsburgh, PA, USA

ABSTRACT

The introduction of machine learning (ML) components in software projects has created the need for software engineers to collaborate with data scientists and other specialists. While collaboration can always be challenging, ML introduces additional challenges with its exploratory model development process, additional skills and knowledge needed, difficulties testing ML systems, need for continuous evolution and monitoring, and non-traditional quality requirements such as fairness and explainability. Through interviews with 45 practitioners from 28 organizations, we identified key collaboration challenges that teams face when building and deploying ML systems into production. We report on common collaboration points in the development of production ML systems for requirements, data, and integration, as well as corresponding team patterns and challenges. We find that most of these challenges center around communication, documentation, engineering, and process, and collect recommendations to address these challenges.

1 INTRODUCTION

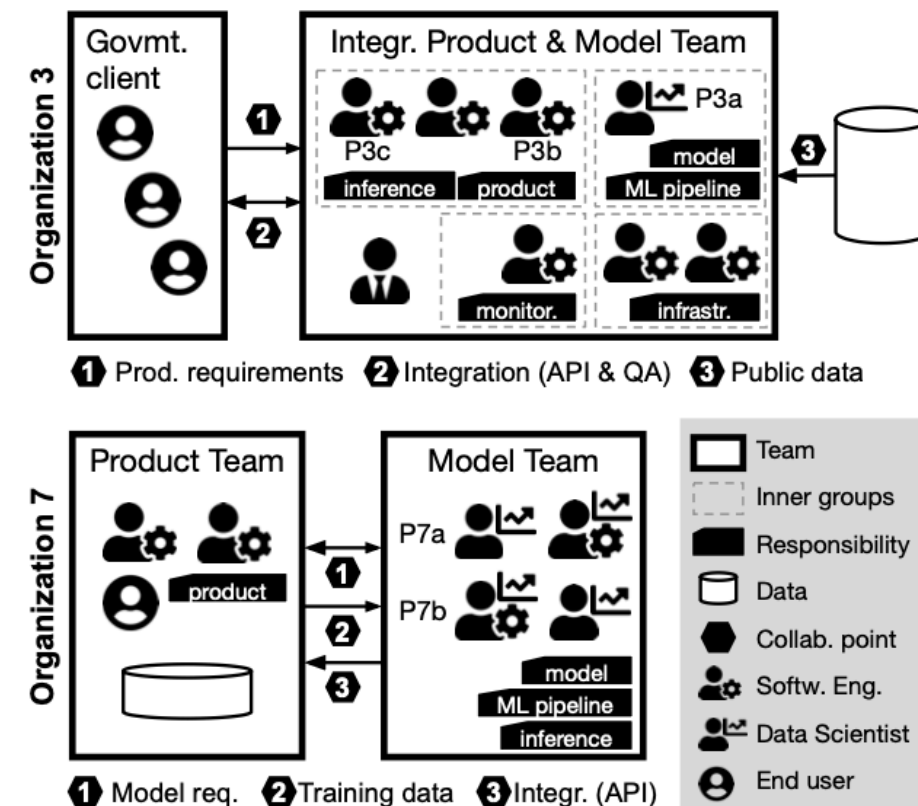


Figure 1: Structure of two interviewed organizations



Collaboration Challenges in Building ML-Enabled Systems

Nahar et al., ICSE 2022

STUDY SNAPSHOT



45
practitioners
interviewed



28
organizations



Qualitative
interview study



Focus: production
ML-enabled
systems



Research question:

Where do software engineers
and data scientists struggle
to collaborate?



Production ML systems are hard not only because of the model, but because requirements, data, integration, and operations create **difficult collaboration boundaries** across roles.



Product
Manager



Data
Scientist



Software
Engineer



Data /
Database



Deployment /
Ops

1 Requirements & Project Planning

- Hard to specify what ML can realistically do
- Model/data requirements are often vague
- Effort estimation is uncertain

2 Training Data

- Data quality and quantity are often inadequate
- Documentation is weak; domain expertise is hard to access
- Training-serving skew causes surprises in production

3 Product-Model Integration

- Responsibilities for engineering and deployment are unclear
- Different cultures and jargon cause misunderstandings
- Testing the whole system is difficult
- Monitoring in production is often missing



Only
11 / 28
organizations
collected
telemetry

CROSS-CUTTING THEMES



Communication



Documentation



Engineering



Process

Most observed problems clustered around these four themes.



Support interdisciplinary teamwork

Increase ML literacy, avoid silos,
involve the right people early



Clarify interfaces and responsibilities

Document expectations for
requirements, data, and APIs



Treat engineering as first-class work

Provide support for
deployment, testing, and MLOps



Plan for quality assurance and monitoring

Design evaluation, telemetry,
and feedback loops from the start



Key takeaway: Successful AI engineering is **socio-technical systems engineering** — not just better models, but better collaboration across the lifecycle.